

2026 年全国大学生计算机设计大赛作品设计报告

作品名称： 基于边缘控制器的多模态 AI 机械臂辅学系统

目 录

摘要 1

1. 作品概述 1

2. 作品设计与实现 9

3. 作品测试与分析 19

4. 创新性说明 23

5. 总结 24

附录 25

摘要

本项目设计并实现了一种基于瑞萨 RA6M5 微控制器的多模态 AI 机械臂辅学系统,面向低龄儿童教育场景,融合计算机视觉、语音交互、边云协同计算及具身智能技术,构建“感知—理解—决策—执行”的闭环智能学习平台。系统通过摄像头、麦克风及多类传感器实现对环境与用户状态的多源信息采集,并基于云端大模型完成语义理解与任务规划,最终由机械臂完成实体操作,实现虚拟认知与物理执行的深度融合。在功能实现方面,系统集成颜色识别与分拣、题目解答、智能光照调节及人脸识别追踪等多种应用场景。其中,视觉模块采用 HSV 颜色空间分割与目标检测算法,实现多目标识别与定位;结合手眼标定与逆运动学求解,实现机械臂对目标物体的精准抓取与分类;在交互层面,引入唤醒词检测与自然语言理解机制,使用户能够通过语音完成任务调用与控制,显著降低操作门槛。在系统架构上,设计基于 Agent 的智能调度框架,将用户语义指令映射为可执行的控制序列,实现多任务动态编排与执行;同时构建前后端分离的 Web 平台,实现任务流程可视化与系统状态监测,提升系统的可观测性与可维护性。

关键词: 边缘计算、具身智能、大模型、多源信息融合、深度学习、实时控制、图像处理

A multimodal AI robotic arm learning system based on Renesas RA6M5

Abstract: This paper presents a multimodal AI robotic arm learning system based on the Renesas RA6M5 microcontroller, designed for early childhood education scenarios. The system integrates computer vision, speech interaction, edge-cloud collaborative computing, and embodied intelligence to construct a closed-loop intelligent framework of “perception–understanding–decision–execution”. Multi-source data from cameras, microphones, and sensors are fused to perceive both environment and user states, while cloud-based large models perform semantic understanding and task planning. The robotic arm executes physical actions accordingly, enabling seamless interaction between virtual intelligence and the physical world.

In terms of functionality, the system supports multiple application scenarios, including color sorting, garbage classification, homework problem solving, adaptive lighting control, and face tracking. The vision module employs HSV-based segmentation and object detection algorithms for multi-target recognition and localization. Combined with hand-eye calibration and inverse kinematics, the system achieves accurate grasping and sorting. For human-computer interaction, a wake-word detection mechanism and natural language understanding are introduced, allowing users to control the system via intuitive voice commands. From an architectural perspective, an Agent-based intelligent scheduling framework is designed to map high-level semantic instructions into executable control sequences, enabling dynamic multi-task orchestration. A web-based front-end and back-end system is also developed for real-time visualization and monitoring, improving system observability and maintainability.

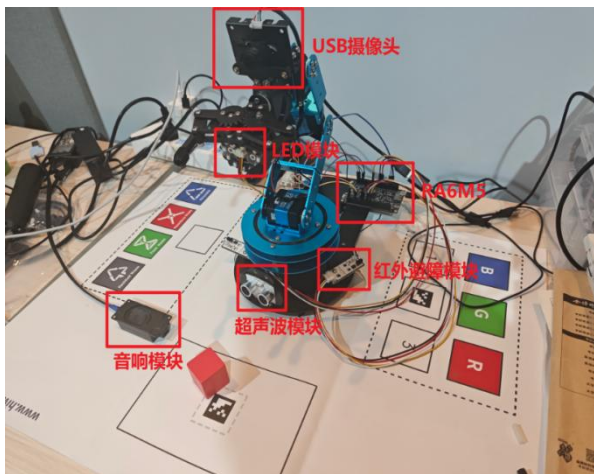
Keywords: Edge computing, embodied intelligence, large model, multi-source information fusion, deep learning, real-time control, image processing

1. 作品概述

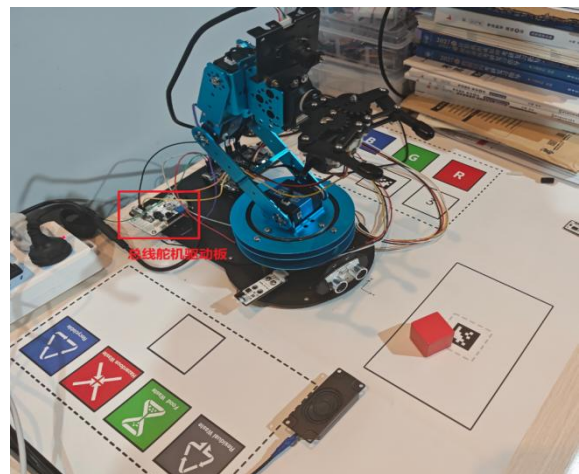
本项目“AI 机械臂辅学系统”面向低龄儿童学习场景，融合人工智能与智能硬件技术，致力于打造一个兼具趣味性与教育意义的互动学习平台。系统以机械臂为核心，通过语音唤醒与自然语言交互，使儿童能够以简单直观的方式参与操作，降低使用门槛并增强学习兴趣。

在功能设计上，系统通过**颜色识别任务**，引导儿童理解基础分类概念，在实践中培养逻辑思维与动手能力；**智能光照调节功能**则帮助儿童认识良好学习环境的重要性。借助 **USB 摄像头与云端大模型**，系统还能对题目进行识别与解答，并通过机械臂动作进行反馈，使抽象知识具象化，提升理解效果。同时，**人脸识别与追踪功能**可辅助家长了解儿童专注状态，培养良好学习习惯。**OLED 屏与 Web 端**展示进一步增强了系统的可视化与互动体验，使儿童能够直观感知任务执行过程。

整个系统通过“唤醒—任务执行—回归初始”的流程设计，帮助儿童建立清晰的操作逻辑与因果认知。该项目不仅提升了学习的趣味性与参与感，也在潜移默化中促进儿童的认知发展、行为规范与科技素养培养。



(a)主控与模块示意图



(b)舵机驱动板示意图

图 1 机械臂模块与 RA6M5 示意图

1.1 项目背景

在人工智能、大模型与具身智能（Embodied AI）快速发展的背景下，教育形态正逐步向低龄化、智能化与沉浸式方向演进。尤其是面向低龄儿童的启蒙教育，传统方式往往依赖单一的图文或口头教学，缺乏互动性与实践性，难以充分激发儿童的学习兴趣与探索能力（如图 2 所示）。因此，构建融合多模态感知、自然交互与实体操作能力的智能辅学系统，成为儿童教育创新的重要方向。



图 2：具身智能系统与传统语言模型能力对比示意图

本项目基于“AI 机械臂辅学系统”理念，融合计算机视觉、语音交互、边云协同计算及大模型推理能力，打造一个具备“看、听、想、做”能力的智能学习伙伴。系统通过唤醒词实现自然交互，例如儿童说出“你好小助手”，即可唤醒系统并发出指令。在颜色识别与分拣任务中，系统可识别不同颜色的积木，并控制机械臂将其分类摆放，帮助儿童建立基础认知能力。系统通过唤醒词驱动的人机交互机制，实现任务级动态编排与执行，并结合机械臂实现物理世界中的精准操作，形成“感知—理解—决策—执行”的闭环，如图 3。颜色识别、智能光照、人脸追踪及题目解答等场景中的应用，不仅提升了学习辅助的智能化水平，也为教育机器人向认知智能体演进提供了实践路径。



图 3 闭环逻辑图

1.2 相关工作

1.2.1 硬件设计

本项目采用瑞萨(Renesas)RA6M5 高性能微控制器作为核心控制单元，构建定制化嵌入式控制系统。RA6M5 基于 Arm Cortex-M33 内核，集成 DSP 与 FPU 单元，具备较强的数据处理能力；同时片上集成丰富的存储资源（Flash、SRAM）及多种外设接口（UART、SPI、IIC、CAN-FD 等），能够满足多传感器数据采集

与多模块协同控制需求，为机械臂运动控制与多模态数据处理提供了可靠的硬件基础。

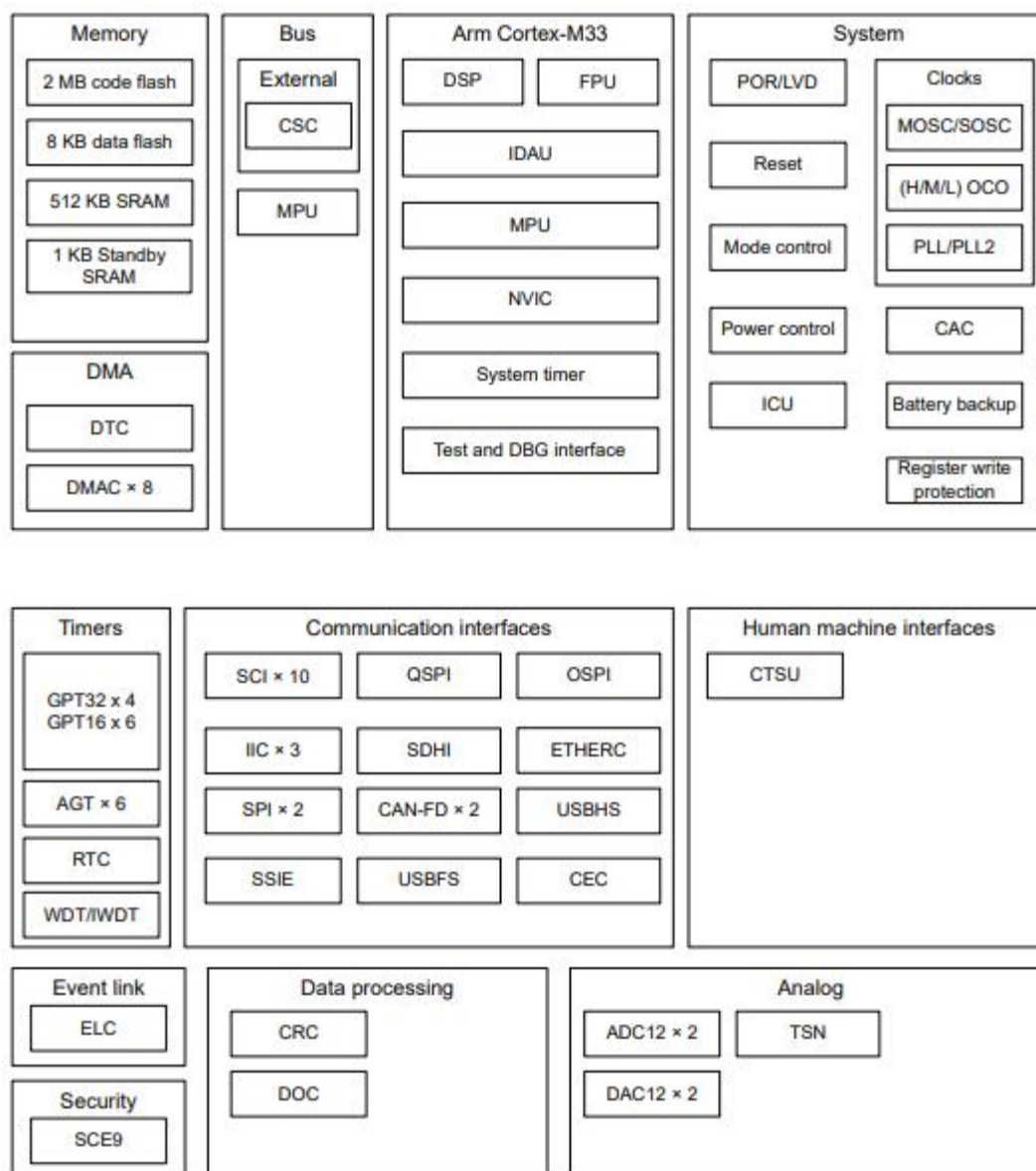
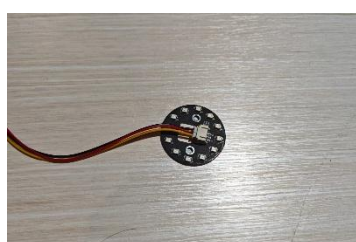


图 4 RA6M5 外设图（来自瑞萨官网数据手册）

具体而言，机械臂驱动系统为串行总线舵机控制板，由 RA6M5 通过总线协议统一调度 6 自由度机械臂，实现高精度、多节点同步控制。其中作为关节的舵机为 LX-225，该舵机最大扭矩为 25kg. cm，只需遵循协议写入舵机特定的控制命令便可返回其当前工作温度、当前电压以及当前位置。

同时，在机械臂前端布置超声波传感器, 见图 5（b），辅助完成目标物体的距离测量与抓取定位；机械臂头部集成 RGB 光源模块, 见图 5（a），实现智能光照调节功能。



(a)



(b)

图 5

超声波传感器使用 CS100 工业级超声波测距芯片内部集成超声波发射电路，超声波接收电路，数字处理电路等，单芯片即可完成超声波测距，测距结果通过脉宽的方式进行输出。CS100 配合使用 40KHZ 的开放式超声波探头，只需要一个 22M Ω 的下拉电阻和 8M 的晶振，即可实现高性能测距功能。原理图如图 6。其中 Trig 为触发信号输入，而 Echo 为回响信号输出。

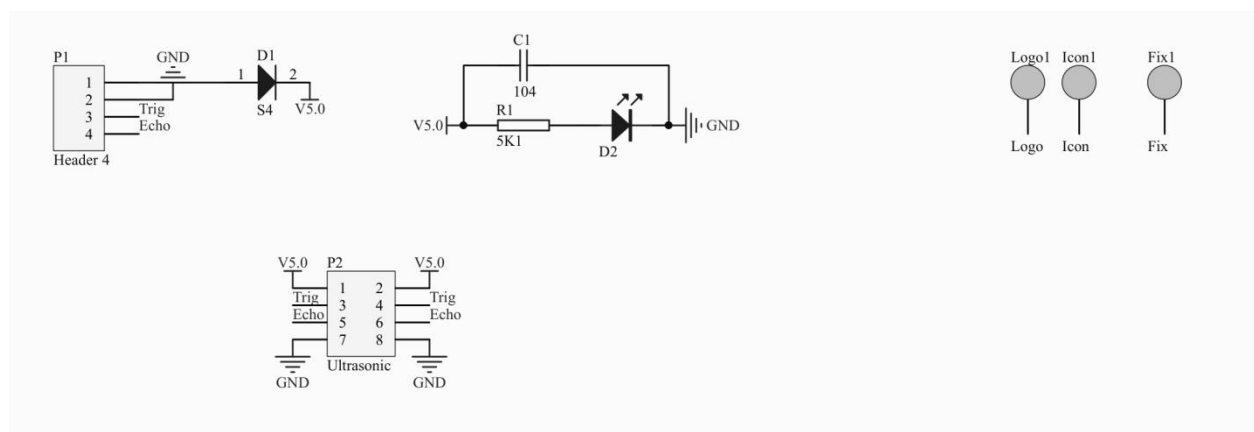


图 6：超声波传感器原理图

红外避障传感器用于检测前方是否存在障碍物的传感器。该传感器具有一个红外发射管和一个红外接收管，一旦传感器遇到障碍物，红外光就会反射回来被接收管接收。其原理图如图 7 所示。当红外接收管未接收到红外光时，信号端 OUT 将输出高电平信号；当接收到红外光后，信号端 OUT 则输出低电平信号。

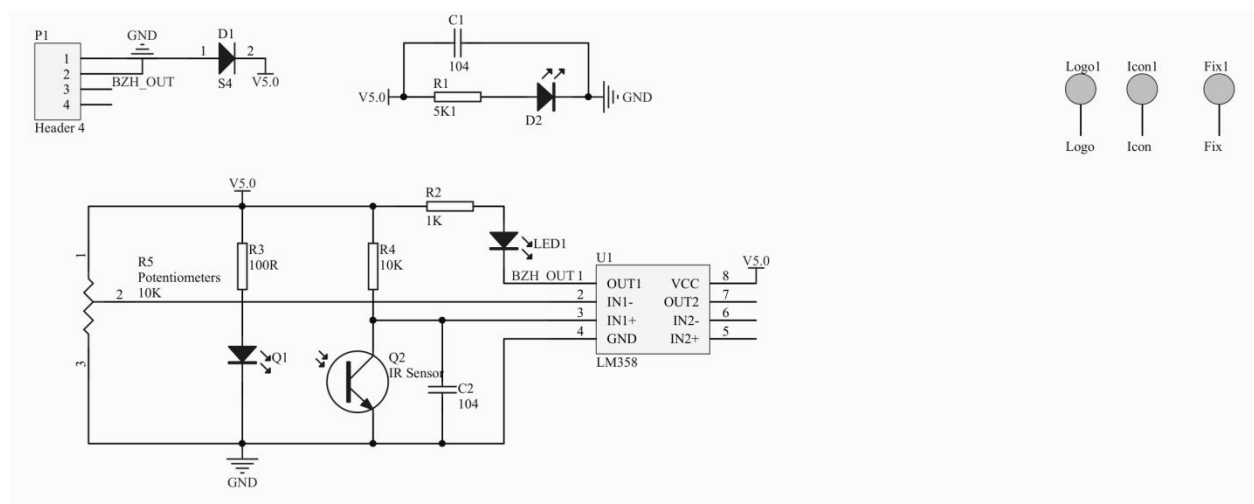


图 7：红外避障传感器原理图

RGB 灯环采用 12 颗 WS2812 RGB 灯珠串联组成，可产生 1000 种以上的实际有效灯色，只需要 1 个 I/O 端口即可控制驱动此模块。其原理图如图 8 所示。DIN 为信号输入，控制 RGB 灯珠颜色与串联数量。虽然其控制协议对时序要求严格，但得益于 RA6M5 有丰富的 32 位定时器外设，因此可以从容应对严格的时序。

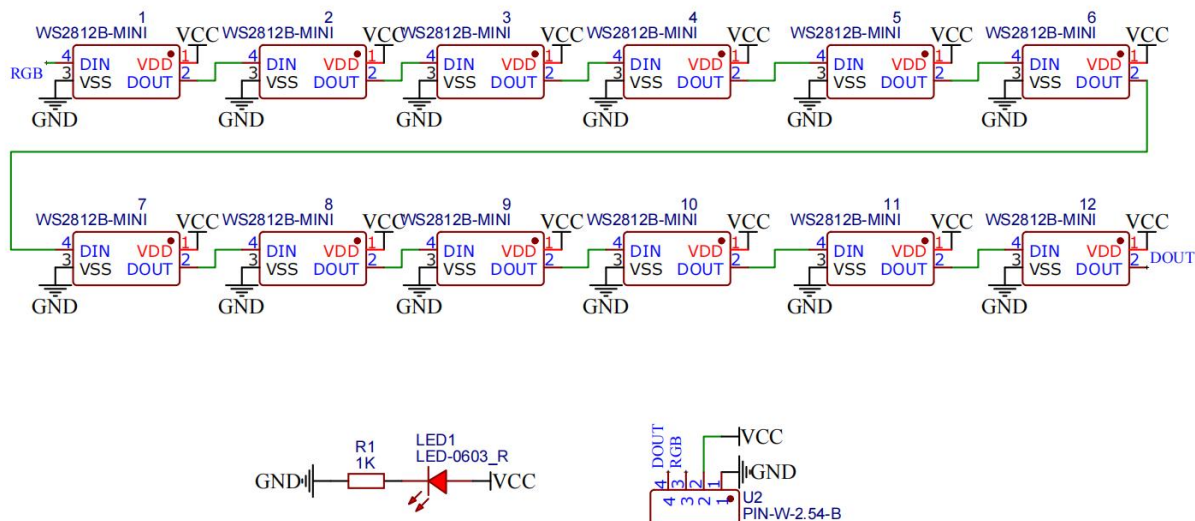
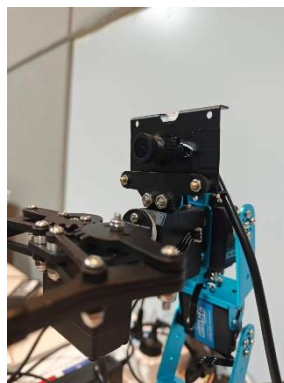


图 8：RGB 灯环原理图

此外，系统还融合了 USB 摄像头、麦克风与扬声器等多模态交互设备（见图 9），构建了面向人机自然交互的感知层与交互层。USB 摄像头用于获取环境图像信息，实现目标识别、定位与跟踪；麦克风模块结合语音唤醒与语音识别算法，实现用户语音指令的输入；扬声器则用于语音反馈与结果播报，增强系统交互体验。通过上述多模态输入与输出设备的协同工作，系统形成了“环境感知—语义理解—决策控制—物理执行—反馈交互”的完整硬件闭环，使用户能够以更加直观、自然的方式与系统进行交互。同时，该硬件架构基于标准接口进行模块化设计，在保证机械臂实时控制性能的前提下，具备良好的扩展性，可灵活接入其他传感器或功能模块，满足不同教育场景下的应用需求，提升系统的通用性与可拓展能力，为实现基于大模型的任务级控制与具身智能应用提供了硬件支撑。



麦克风和扬声器设备

“眼在手上”摄像头

图 9

1.2.2 软件设计

在软件层面，现有教育机器人多采用规则驱动或简单脚本逻辑，缺乏复杂任务理解与动态调度能力。为此，本文引入基于 Agent 架构的智能控制系统，通过接入云端大模型，实现任务级语义理解与决策规划。

系统核心为一个统一的智能 Agent 模块，负责解析用户语音指令（在唤醒词触发后），并结合上下文信息调用云端大模型进行意图识别与任务拆解，从而动态选择对应功能模块（如颜色分拣、题目解答等）。在执行层面，Agent 将高层语义指令转化为具体控制序列，下发至底层嵌入式系统驱动机械臂动作。此外，系统设计并实现了一套前后端分离的 Web 应用平台。前端用于可视化展示机械臂当前状态、任务执行流程及系统反馈信息；后端则负责任务调度、日志记录及与云端模型的通信。该 Web 系统不仅增强了系统的可观测性，也为后续功能扩展与远程运维提供了基础。

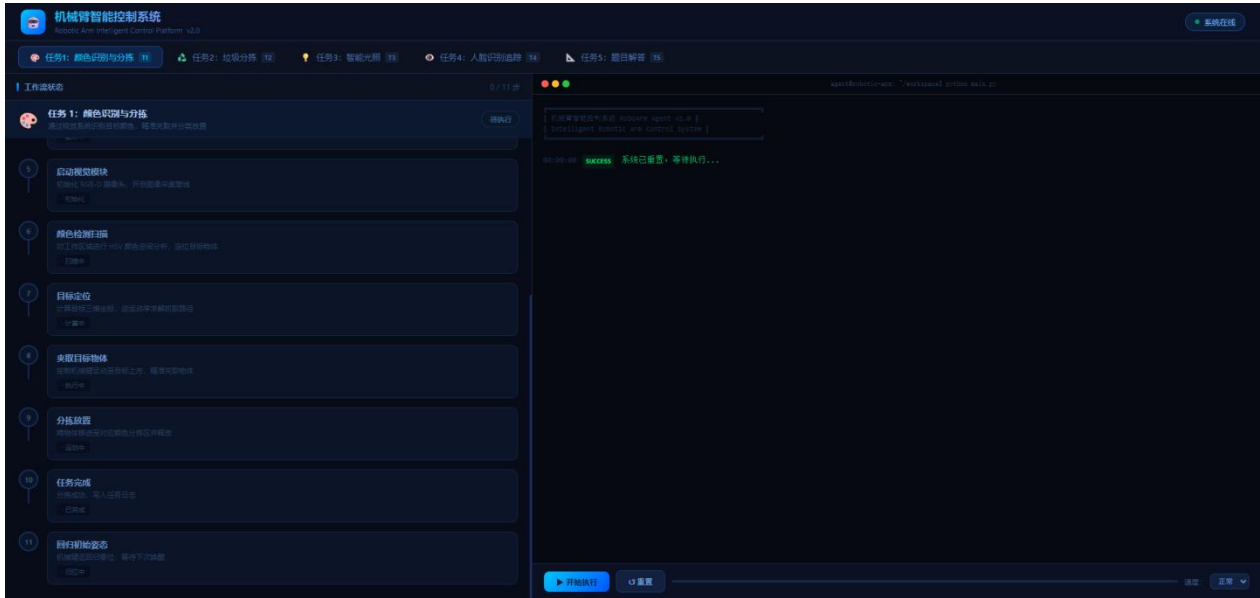


图 10：机械臂智能控制系统 Web 平台界面及任务执行流程可视化示意图

1.2.3 算法设计

在算法层面，现有研究多聚焦于单一功能模块的优化，如目标识别、路径规划或语音交互等，缺乏面向多任务融合场景的统一算法框架。针对这一问题，本文面向复杂任务协同与连续执行需求，构建了融合视觉感知、语义理解、任务规划与运动控制的多模块协同算法体系。通过模块间的信息耦合与流程联动，实现从多源感知输入到高层语义决策，再到机械臂精确执行的闭环处理机制，从而提升系统整体智能水平与任务执行效率。首先，在机械臂控制方面，系统实现了正向运动学与逆向运动学求解算法，用于完成从关节空间到任务空间的映射，以及目标位置到舵机角度的反解计算（如图 11 所示），从而保证机械臂运动的精确性与可控性。

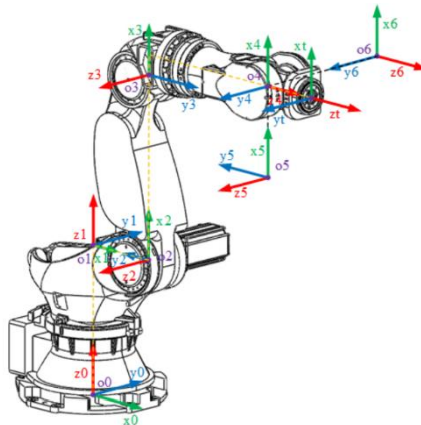


图 11：机械臂运动学建系示意图

其次，在视觉处理方面，引入相机去畸变算法对 USB 摄像头采集图像进行校正，提高后续识别精度；同时，通过手眼标定算法建立机械臂坐标系与相机坐标系之间的映射关系，实现视觉引导下的精准抓取，如图 12 所示。

在抓取策略上，系统融合超声波传感器的距离信息，设计基于多传感器融合的夹取算法，有效提升抓取成功率与稳定性。



图 12：用于相机标定与手眼标定的棋盘格标定板和 ArUco 标记

在人机交互方面，系统集成轻量化唤醒词检测模型，实现低功耗实时语音唤醒；同时结合人脸识别与目标跟踪算法，对学生在位情况进行检测与动态跟踪，为学习行为分析提供支持。

1.3 特色描述

本项目围绕“AI 机械臂辅学系统”，在低龄儿童教育场景下，融合具身智能、多模态交互与大模型能力，形成了具有鲜明特色的智能教育解决方案：

首先，在系统架构上，项目实现了“感知—认知—决策—执行”的闭环设计。通过摄像头、麦克风、超声波等多传感器协同感知环境与用户状态，结合云端大模型进行语义理解与任务规划，最终由机械臂完成实体操作，打通虚拟智能与物理世界之间的交互链路，体现了具身智能在教育领域的落地应用，如图 13 所示。

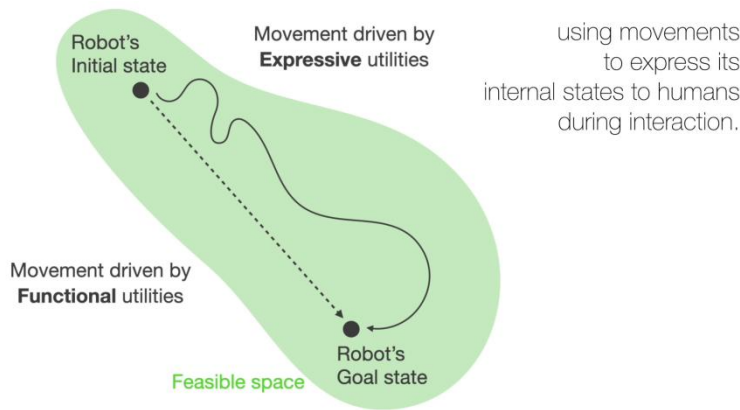


图 13：机器人在功能目标与表达性需求共同驱动下的运动规划示意图

其次，在交互方式上，系统采用唤醒词驱动的自然语言交互模式，显著降低了低龄儿童的使用门槛。儿童无需复杂操作，仅通过语音即可完成任务调用与控制，使学习过程更加自然、直观。同时结合人脸识别与行为检测，实现对儿童学习状态的实时感知与反馈，增强系统的主动性与陪伴感。

在功能设计上，系统以“寓教于做”为核心理念，将颜色分拣等任务转化为可操作的实体交互过程，帮助儿童在动手实践中建立认知模型。同时引入题目识别与大模型讲解能力，使系统不仅具备操作性，还具备认知辅导能力，实现“动手+动脑”的双重提升。

在工程实现上，项目采用瑞萨 RA6M5 作为核心控制单元，设计了总线舵机控制板，实现高精度、多自由度机械臂控制，并结合多传感器融合算法提升系统稳定性与鲁棒性。同时构建 Web 可视化平台，对任务流程与系统状态进行实时展示，增强系统的可解释性与可维护性。

可见，本项目在技术融合、交互体验与教育价值三个层面均具备显著特色，为低龄儿童智能教育提供了一种兼具创新性与实用性的解决方案。

1.4 应用前景

随着人工智能、大模型与智能硬件的不断发展，教育领域正加速向智能化与个性化方向转型。“AI 机械臂辅学系统”作为融合具身智能与多模态交互的创新载体，在多个场景中展现出广阔的应用前景。

首先，在基础教育领域，尤其是幼儿园与小学阶段，该系统可作为智能教学辅助工具，融入课堂教学与课后实践环节。**通过颜色分拣等互动任务，提升儿童的动手能力与认知水平；结合语音交互与题目讲解功能，实现个性化学习辅导，缓解教师在大班教学中的压力。同时，人脸识别与专注度检测功能可辅助教师了解学生状态，优化教学管理。**

其次，在家庭教育场景中，该系统可作为智能陪伴型学习助手，为儿童提供全天候的学习支持与互动体验。**家长可通过 Web 端远程查看学习情况，系统则通过智能光照、行为提醒等功能营造良好的学习环境，提升家庭教育质量。**

此外，在科普教育与创新实践领域，该系统具备良好的展示与教学价值。其融合机械臂控制、计算机视觉与人工智能算法，可作为中小学 STEM 教育或创客教育的重要平台，帮助学生理解前沿技术原理，培养创新思维与工程实践能力。

在更广泛的层面，该系统所构建的“**AI+机械臂+教育**”模式，也可拓展至特殊教育、智慧教室建设及教育机器人产业中，具有良好的产业化潜力与推广价值。未来，随着算法与硬件的进一步优化，该类系统有望成为智能教育生态中的重要组成部分。

2. 作品设计与实现

2.1 系统方案

2.1.1 颜色分拣功能

颜色识别与分拣任务以机载 USB 摄像头作为主要视觉感知器件,对工作台区域进行连续图像采集,将画面从 RGB 色彩空间转换至 HSV 空间,依据目标颜色的色调与饱和度区间精确定位目标物体的二维像素坐标。与此同时,安装于机械臂底座的超声波传感器实时采集目标物体与机械臂之间的距离信息,结合摄像头的视角参数完成目标的空间位置解算,从而获取可用于运动规划的三维参考坐标。系统随后对机械臂各关节进行逆运动学求解,生成平滑无碰撞的抓取路径,控制机械臂移动至目标正上方并闭合夹爪完成抓取动作,最终将物体移送至与其颜色对应的分拣区域并精准释放,实现全流程自动化分拣,如图14。

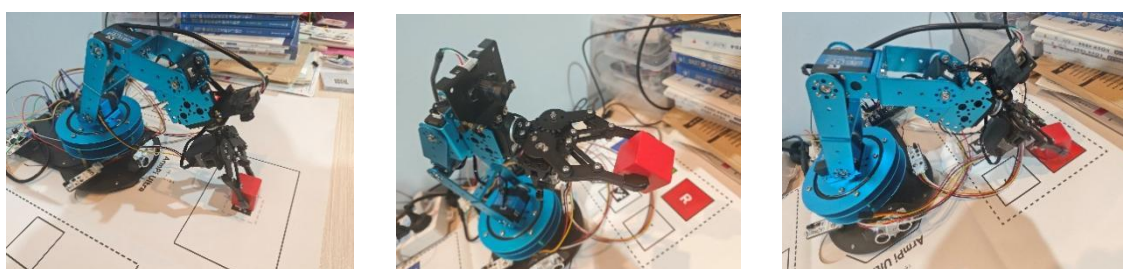


图14：机械臂实现自动化分拣示意图

机械臂运动的同时,web 程序接受来自 RA6M5串口返回的数据,将执行任务的工作流和对应的终端信息渲染在 web 端,如图15。

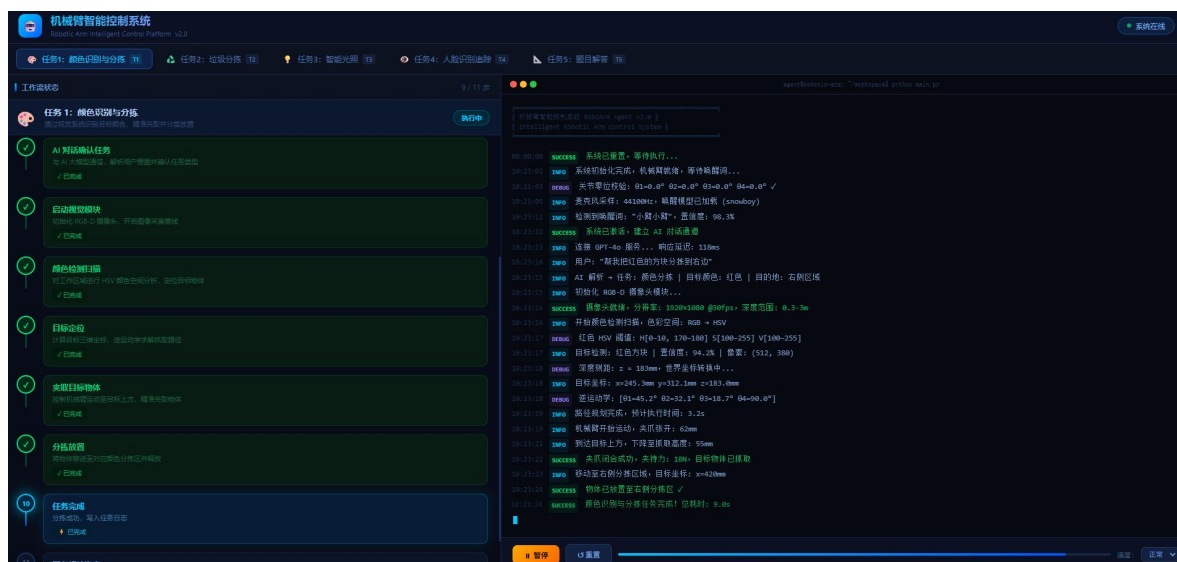


图15：机械臂颜色分拣任务执行过程与实时日志输出界面

2.1.2 作业解答功能

题目解答任务充分发挥机械臂的空间调姿能力与 AI 大模型的强大推理能力,为学生提供智能化的学习辅助支持。任务开始后,机械臂自动将摄像头调整至最优俯视采集角度并触发自动对焦,对工作台上的纸质题目进行清晰图像采集,随后调用 OCR 引擎对图像中的印刷文字、手写内容及

数学公式进行精准识别，将视觉信息还原为结构化的文本与公式表达。系统对题目内容进行类型解析，构建语义完整的提示词并发送至云端 AI 大模型，模型结合学科知识图谱与逻辑推理能力生成包含详细解题步骤与最终答案的完整解答。解答内容经语音合成引擎转化为自然流畅的语音输出，帮助学生理解解题思路而非仅获取答案，真正实现寓教于用的智能辅学效果。每次解答记录均自动存入题目历史库，支持后续复习与统计分析，如图19所示。

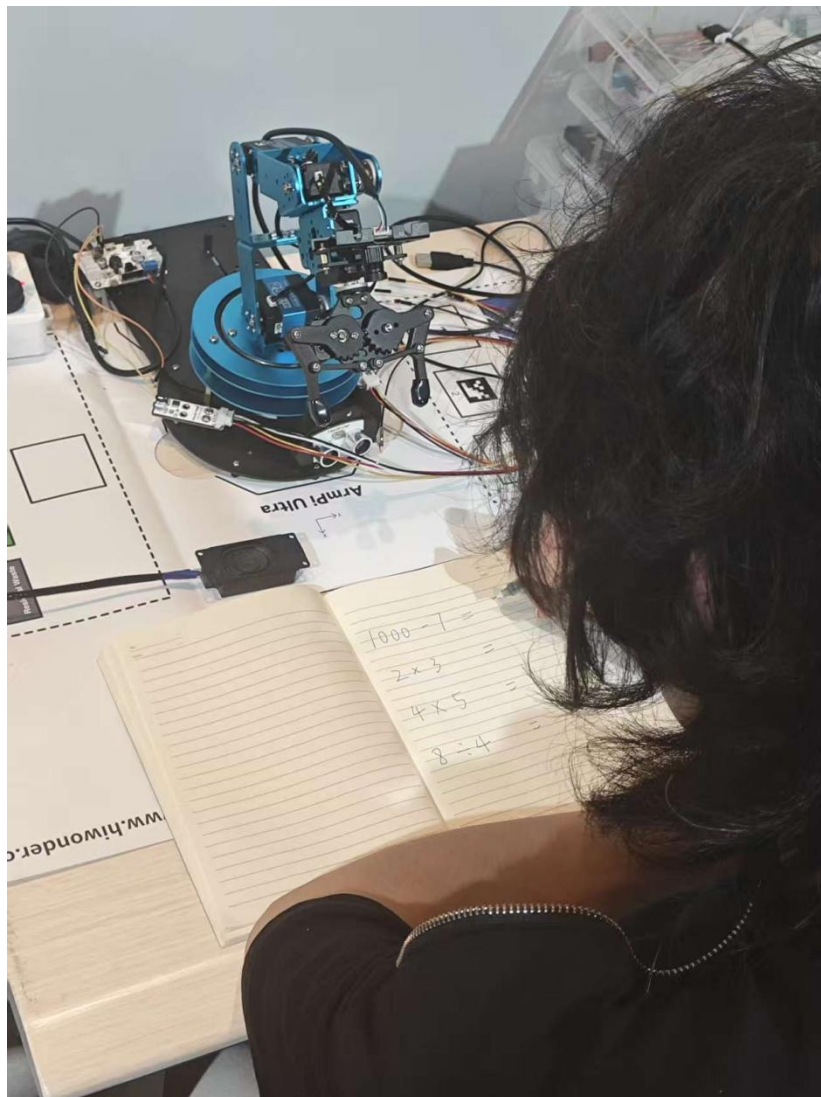


图19：机械臂实现作业解答示意图

web 端对整个学习过程进行监视，并打印机械臂运行结果和大模型生成答案，如图20所示。

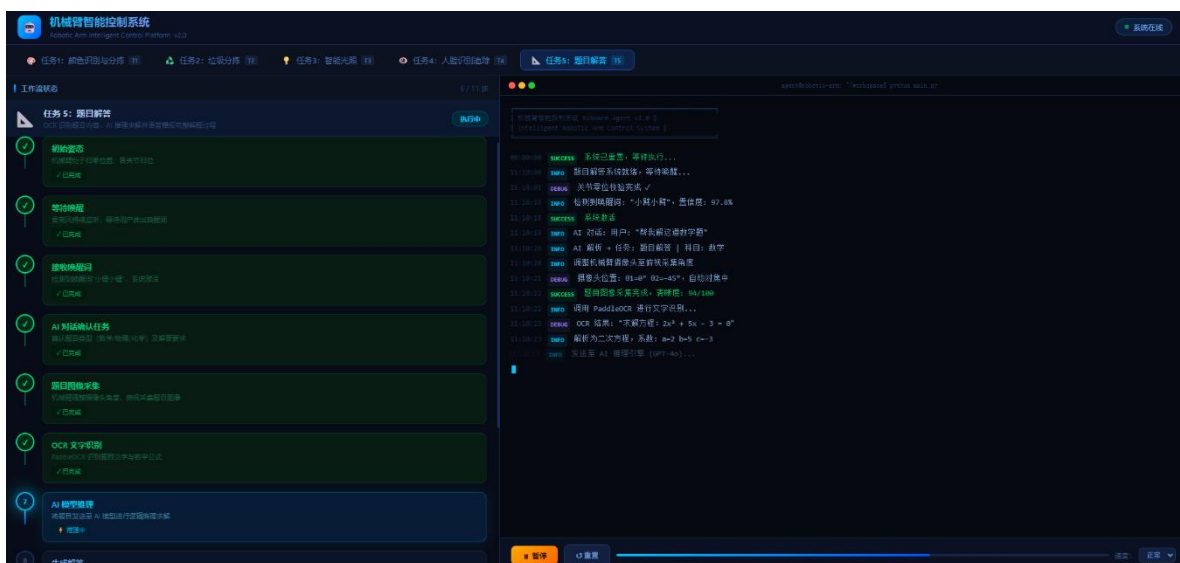


图20：机械臂作业解答任务执行过程与实时日志输出界面

2.1.3 智能光照功能

智能光照任务将环形 LED 灯组固定安装于机械臂头部，由机械臂的空间姿态灵活调配光源的照射方向与覆盖范围，为工作区域提供精准补光。学生无需操作任何按钮，仅需以日常口语化方式向系统表达光照感受，例如“现在太亮了”“光线有点暗”“换成暖一点的颜色”等自然语言描述，云端 AI Agent 接收到语音指令后对用户意图进行语义解析，将抽象的主观感受转化为具体的亮度值与色温参数。解析结果通过网络下发至搭载 RA6M5 微控制器的嵌入式控制板，由控制板输出对应占空比的 PWM 信号，驱动 LED 驱动芯片实时调节灯光亮度与色温，形成“自然语言输入 → 云端语义解析 → 嵌入式 PWM 输出”的完整闭环控制链路，使光照调节过程对用户完全透明且极为便捷，如图21所示。

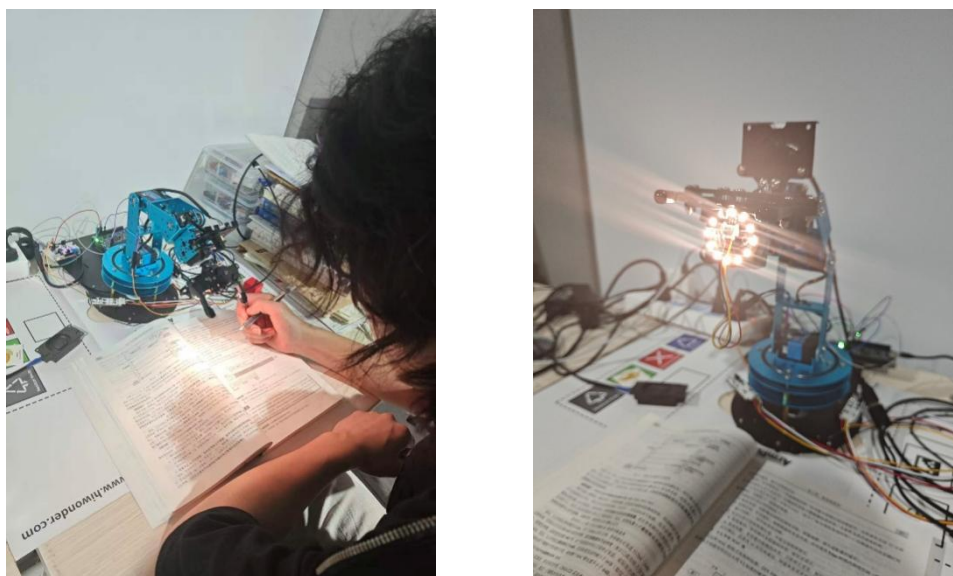


图21：机械臂实现智能光照示意图

此功能将机械臂赋予台灯的功能，可以使学生充分的感受到科技的创造性。我们的 web 端同样会对机械臂和 LED 的状态进行监测。左侧为该任务的工作流，右侧为终端的打印信息，如图22所示。

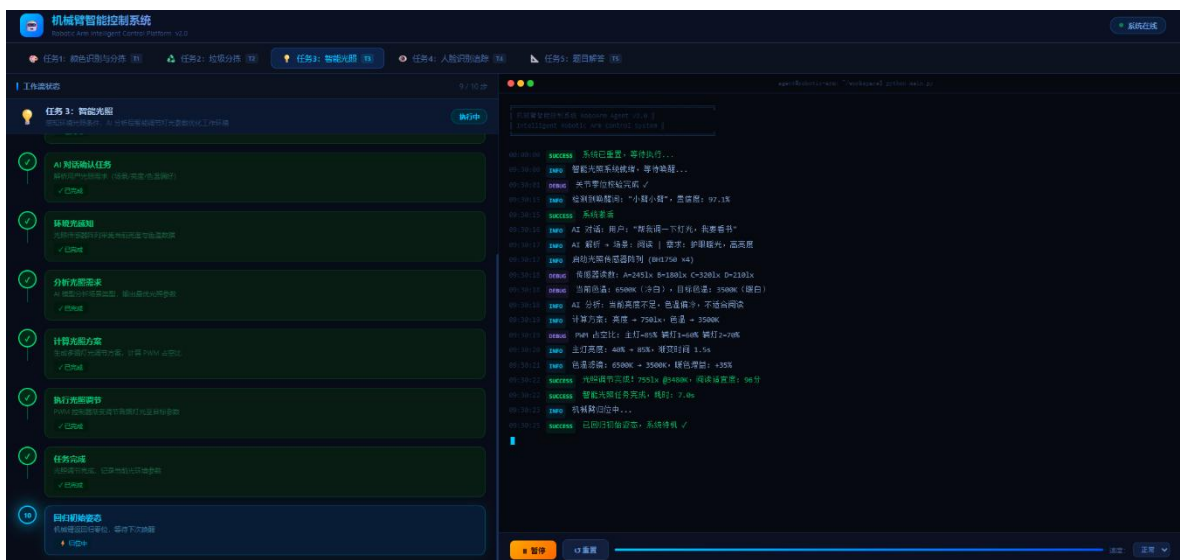
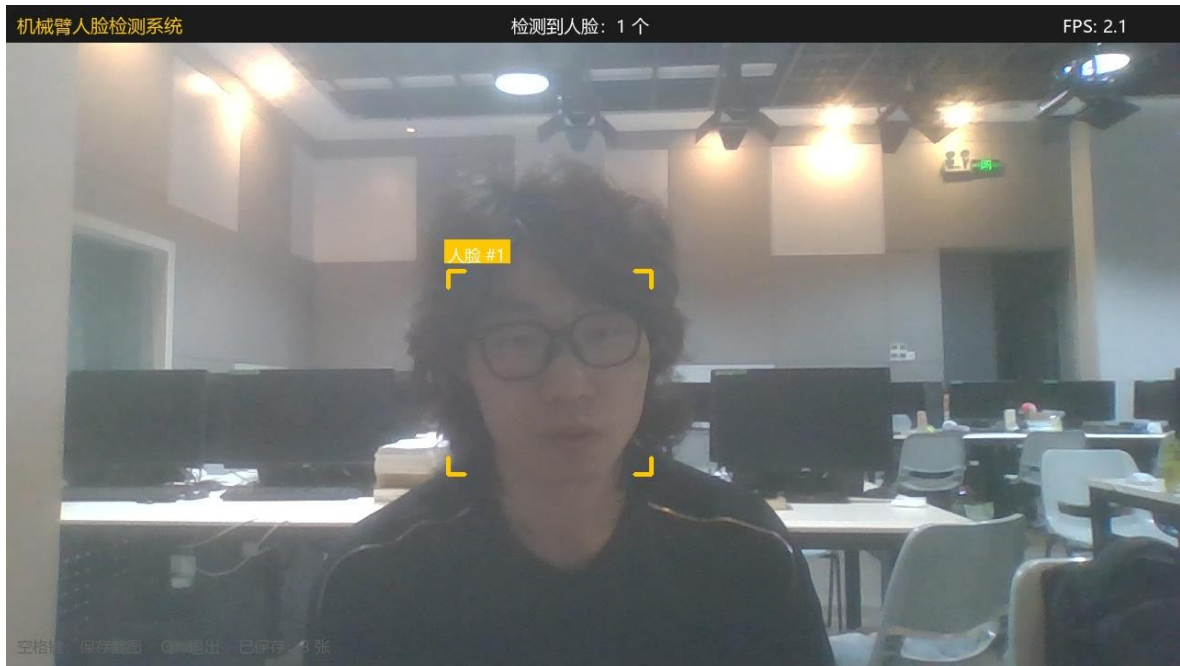


图22: 机械臂智能光照执行过程与实时日志输出界面

2.1.4 人脸追踪功能

人脸识别追踪任务面向课堂管理与学习行为监督场景，核心目的在于实时判断学生是否在座位上专注学习，为教师或智能班级管理系统提供客观的出勤与注意力状态数据。系统通过机载摄像头持续采集工作区域的视频流，人脸检测模型对每帧画面进行实时推理，识别画面中出现的人脸数量与位置。检测到人脸后，特征提取模块生成高维人脸特征向量，与预先录入的学生人脸数据库进行比对，完成身份确认。目标锁定后，控制器根据人脸中心与画面中心的偏差量实时计算关节角度补偿值，驱动机械臂平滑旋转追踪目标，确保被监测学生始终处于摄像头有效视野内。系统同步记录识别结果与追踪状态，支持生成学生在岗时长统计与异常离座提醒，为智能化教学管理提供数据支撑，如图23所示。



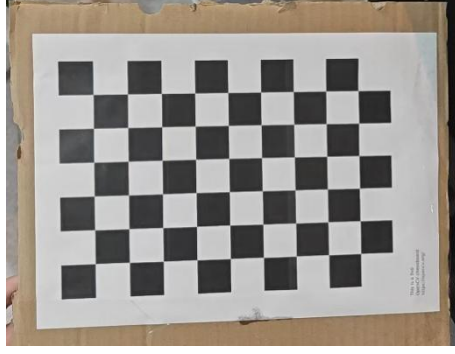


图25：用于进行标定的棋盘格

在标定过程中，首先提取每张图像中的棋盘格角点，并建立棋盘格平面世界坐标系与图像像素坐标系之间的对应关系。设空间点在相机坐标系下的坐标为 (X_c, Y_c, Z_c) ，则其归一化成像坐标可表示为：

$$x = \frac{X_c}{Z_c}, \quad y = \frac{Y_c}{Z_c} \quad (1)$$

理想针孔模型下，归一化坐标与像素坐标之间的关系为：

$$u = f_x x + c_x, \quad v = f_y y + c_y \quad (2)$$

其中， f_x 和 f_y 分别表示图像在 x 与 y 方向上的等效焦距， (c_x, c_y) 为主点坐标。实际成像过程中，由于镜头制造误差及安装误差，图像会产生径向畸变和切向畸变。径向畸变主要表现为桶形畸变或枕形畸变，其数学模型可写为：

$$r^2 = x^2 + y^2$$

$$x_r = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6), \quad y_r = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \quad (3)$$

其中， k_1 、 k_2 和 k_3 为径向畸变系数， r 表示图像点到主点的归一化径向距离。一般来说，距离主点越远，径向畸变越明显，因此图像边缘和角落区域的采样对径向畸变估计尤为重要。

切向畸变主要来源于镜头光轴与成像平面不完全平行，其模型可表示为：

$$x_t = 2p_1 xy + p_2(r^2 + 2x^2), \quad y_t = p_1(r^2 + 2y^2) + 2p_2 xy \quad (4)$$

其中， p_1 和 p_2 为切向畸变系数。综合径向畸变与切向畸变后，畸变归一化坐标可表示为：

$$x_d = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2) \quad (5)$$

$$y_d = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy \quad (6)$$

进一步映射到像素坐标系后，可得畸变图像点坐标：

$$u_d = f_x x_d + c_x, \quad v_d = f_y y_d + c_y \quad (7)$$

在标定求解时，通过最小化棋盘格角点的实际检测位置与模型重投影位置之间的误差，估计摄像头内参数及畸变参数。其目标函数通常表示为：

$$\min \sum_{i=1}^N |\mathbf{m}_i - \widehat{\mathbf{m}}_i|^2$$

其中， $\mathbf{m}_i$ 为第 i 个实际检测到的角点像素坐标， $\widehat{\mathbf{m}}_i$ 为根据当前相机模型计算得到的重投影坐标， N 为所有标定图像中角点总数。通过对该目标函数进行优化，可得到摄像头的焦距、主点位置以及径向和切向畸变系数。

我们的项目采用 **45 张** 分布于图像不同区域的棋盘格图像进行标定，其中四个角落区域与中心区域均有充分采样。这种采样方式增强了角点在整个视场范围内的分布均匀性，尤其提高了边缘区

域和角落区域的约束能力，因此能够更有效地估计镜头畸变，提升手眼标定和机械臂定位计算的精度。



图26：已标定的棋盘格

2.2.2 计算手眼标定矩阵？

在完成摄像头内参及畸变参数标定后，为实现视觉坐标系与机械臂坐标系之间的转换关系，需要进一步进行手眼标定。手眼标定的目标是求解相机坐标系与机器人末端执行器坐标系之间的刚性变换关系。

我们的项目采用经典的 $AX = XB$ 模型进行手眼标定建模。设机器人末端在基坐标系下的位姿变换为 A ，标定板在相机坐标系下的位姿变换为 B ，则手眼标定问题可表示为：

$$A_i X = X B_i \quad (8)$$

其中：

- X ：待求的手眼变换矩阵（相机坐标系到末端执行器坐标系）
- A_i ：机器人末端在基坐标系下的相对运动
- B_i ：标定板在相机坐标系下的相对运动

每个位姿均可表示为齐次变换矩阵：

$$T = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \quad (9)$$

$$R \in SO(3), \quad t \in \mathbb{R}^3$$

其中： $R \in SO(3)$ 为旋转矩阵， $t \in \mathbb{R}^3$ 为平移向量

对于第 i 次和第 j 次采样，其相对运动为：

$$\begin{aligned} A_{ij} &= T_{base}^{end}(j)^{-1} T_{base}^{end}(i) \\ B_{ij} &= T_{cam}^{target}(j) T_{cam}^{target}(i)^{-1} \end{aligned} \quad (10)$$

将旋转部分从 $AX = XB$ 中分离：

$$R_A R_X = R_X R_B$$

可转化为：

$$R_A R_X R_B^{-1} = R_X$$

这里我们使用 opencv 内置的四元数方法进行求解。

在已求得旋转矩阵 R_X 后，平移部分满足：

$$R_A t_X + t_A = R_X t_B + t_X$$

整理可得：

$$(R_A - I)t_X = R_X t_B - t_A$$

通过多组数据构建超定方程组，采用最小二乘法求解 t_X 。

对于多组采样数据，可构建误差函数：

$$\min \sum_i |A_i X - X B_i|_F^2$$

其中 $\|\cdot\|_F$ 表示 Frobenius 范数。

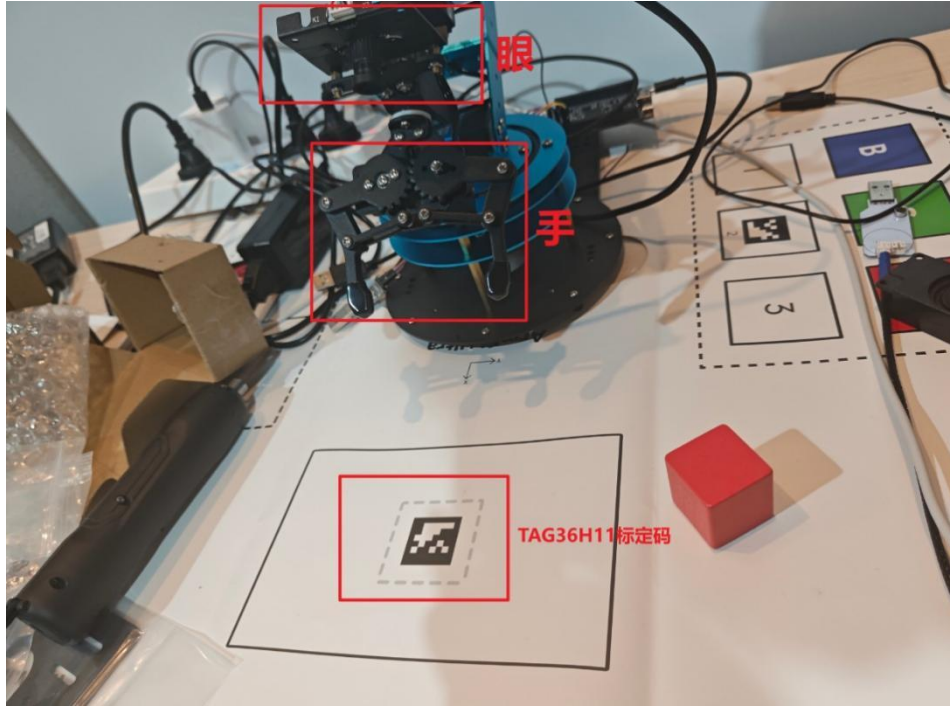


图27：机械臂手眼协同系统与 TAG36H11 标记定位实验场景

2.2.3 机械臂正解算

为实现机械臂末端执行器在空间中的精确定位，需要建立关节空间到笛卡尔空间的映射关系，即机械臂正运动学模型。正运动学的目标是在已知各关节变量的情况下，求解末端执行器在基坐标系下的位姿。

我们的机械臂采用 Denavit-Hartenberg (D-H) 方法进行建模。对于第 i 个关节，其坐标系之间的变换可表示为：

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (11)$$

其中：

$$\theta_i, \quad d_i, \quad a_i, \quad \alpha_i$$

分别表示关节角、连杆偏移、连杆长度以及连杆扭转角。

机械臂末端执行器相对于基坐标系的位姿可通过各关节变换矩阵连乘得到：

$$T_{base}^{end} = T_1^0 T_2^1 \dots T_n^{n-1}$$

该矩阵可表示为：

$$T_{base}^{end} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \quad (12)$$

末端执行器在基坐标系下的位置可表示为：

$$p = [x \ y \ z]^T$$

末端姿态由旋转矩阵表示，进一步转换为欧拉角形式：

$$\phi = \arctan 2(r_{32}, r_{33}), \quad \theta = \arctan 2\left(-r_{31}, \sqrt{r_{32}^2 + r_{33}^2}\right), \quad \psi = \arctan 2(r_{21}, r_{11})$$

其中 r_{ij} 为旋转矩阵中的元素。

在实际计算中，通过读取机械臂各关节的角度值 θ_i ，结合预先标定的 D-H 参数，逐级计算各关节之间的变换矩阵，并进行矩阵连乘，从而得到末端执行器在基坐标系下的位姿。

该过程为：

$$T_{base}^{end} = f(\theta_1, \theta_2, \dots, \theta_n)$$

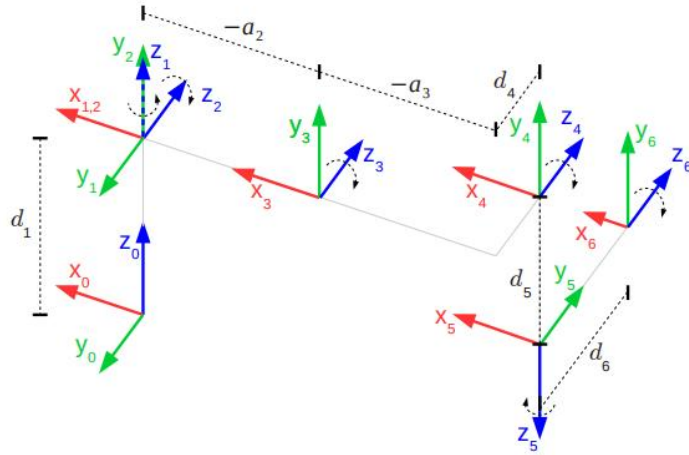


图28：机械臂各关节坐标系分配及 D-H 参数建模示意图

2.2.4 机械臂逆解算

机械臂逆运动学的目标是在已知末端执行器位姿的情况下，求解对应的关节变量。与正运动学不同，逆运动学通常具有多解性和非线性特性，是机械臂控制中的关键问题。我们团队的机械臂是5自由度加上末端夹爪。

设末端执行器在基坐标系下的目标位姿为：

$$T_{base}^{end} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix}$$

逆运动学问题可表示为：

$$f(\theta_1, \theta_2, \dots, \theta_n) = T_{base}^{end} \quad (13)$$

即求解关节变量：

$$\theta_1, \theta_2, \dots, \theta_n$$

首先计算前三个关节对应的旋转矩阵：

$$\mathbf{R}_0^3 = \mathbf{R}_z(\theta_1)\mathbf{R}_y(\theta_2)\mathbf{R}_y(\theta_3)$$

则腕部旋转矩阵为：

$$\mathbf{R}_3^6 = (\mathbf{R}_0^3)^T \mathbf{R}$$

根据旋转矩阵元素可求得后三个关节角：

$$\theta_4 = \arctan 2(r_{32}, r_{33})$$

$$\theta_5 = \arctan 2\left(\sqrt{r_{32}^2 + r_{33}^2}, r_{31}\right)$$

$$\theta_6 = \arctan 2(r_{21}, r_{11})$$

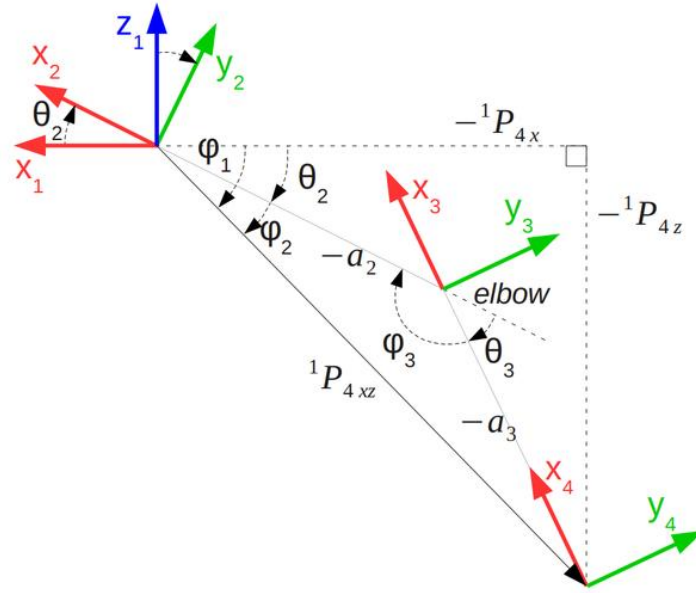


图29：机械臂逆解建模示意图

2.3 硬件框图

本系统围绕机械臂智能控制任务，构建了由感知层、控制层与执行层组成的分层式硬件架构，并形成“感知—决策—执行—反馈”的闭环控制系统。

在系统总体结构上，硬件平台划分为感知层、控制层和执行层三个主要部分。感知层负责环境信息采集，控制层完成数据处理与任务决策，执行层实现具体动作输出，各层之间通过标准接口协同工作，形成完整的闭环控制流程。

感知层作为系统输入端，主要包括USB摄像头、麦克风及其他辅助传感器。其中，USB摄像头用于目标检测与空间定位，为机械臂提供视觉信息；麦克风用于语音指令采集，实现人机交互控制；多种传感器则用于补充环境状态信息。通过多模态信息融合，提高系统对复杂场景的感知能力与鲁棒性。

控制层以主控芯片RA6M5为核心，承担系统的计算与控制任务。该层主要完成图像数据处理、任务调度及控制决策等功能。通过UART、SPI、IIC等通信接口，实现与各类外设的数据交互；同时结合定时器、DMA及中断等片上资源，实现高效的数据传输与实时控制。此外，系统支持与上位机或云端模型通信，将高层语义任务转化为具体控制指令，从而提升系统智能化水平。

执行层作为系统输出端，主要包括机械臂本体、电机驱动模块及末端执行器。控制层根据任务规划结果输出PWM等控制信号，驱动各关节电机完成预定运动，实现目标物体的抓取与放置等操作。机械臂多关节结构保证了系统具备一定的空间操作能力。

在交互与反馈方面，系统集成扬声器用于语音反馈输出，同时结合Web可视化平台，实现系统状态展示、任务流程监控及日志记录等功能，从而构建完整的人机交互闭环，提高系统的可观测性与可操作性。

在系统支撑方面，设计了电源管理模块，为各功能单元提供稳定可靠的电源供应，并通过合理的供电分配与稳压设计，保障系统长期稳定运行。

综上所述，该硬件系统具有多模态融合、实时控制能力强、模块化程度高及良好的扩展性等特点，能够有效支撑机械臂智能控制与复杂任务执行需求，并为后续功能扩展与远程运维提供了可靠基础。

3. 作品测试与分析

3.1 测试环境搭建

测试平台以 Renesas RA6M5（R7FA6M5BF，Cortex-M33 内核）为核心控制器，搭载 6 路串口总线舵机（兼容 LeArm 协议，ID 1~6）及 I²C 接口步进电机直线滑台（设备地址 0x35，8 细分模式）。具体硬件配置如下：

1) 机械臂本体：4 自由度关节臂（基座旋转、肩部俯仰、肘部俯仰、腕部俯仰）+ 腕部旋转舵机 + 夹爪舵机，共 6 路舵机；舵机控制精度 4.167 步/度，位置行程 125~875（对应约 180°）。

2) 传感器配置：USB 摄像头（分辨率 1280×720，帧率 30fps，安装于机械臂末端上方，倒置固定）；超声波测距模块；OLED 显示屏。

3) 上位机：PC（Windows 10/11，Python 3.10），通过 USB 转串口模块与 MCU 通信，波特率 115200bps。

4) 调试工具：Keil MDK（μVision）+ J-Link 仿真器，Renesas FSP（Flexible Software Package）配置外设。

测试工作台搭建如下：桌面铺设测试场景（颜色分拣区域），固定 AprilTag 36h11 标记贴纸（边长 2.5 cm）用于手眼标定，机械臂底座固定于工作台中央，USB 摄像头连接至上位机，如图30

所示。

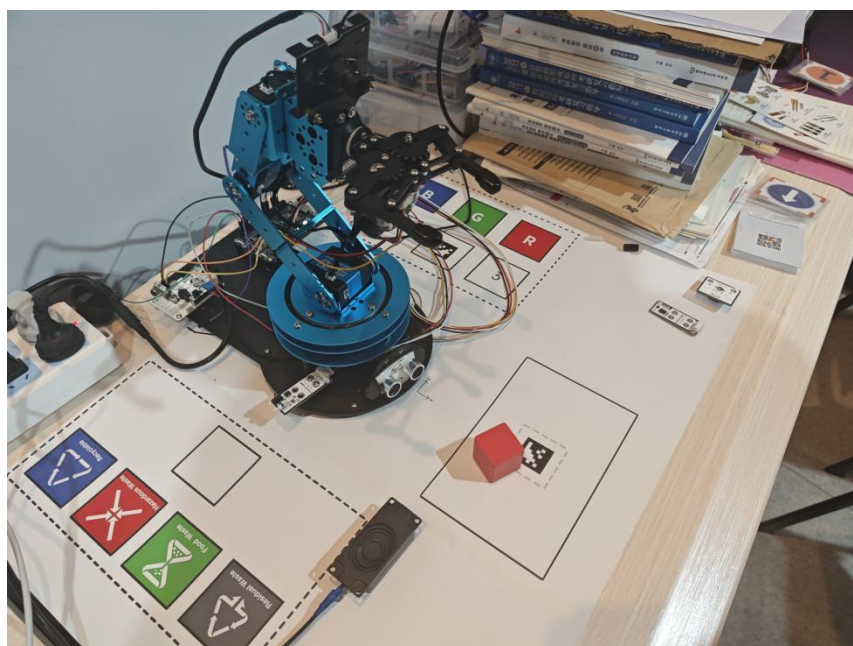


图30：机械臂逆解建模示意图

类别	工具/版本
嵌入式开发	Keil MDK μ Vision 5, Renesas FSP 库
上位机语言	Python 3.11
图像处理	OpenCV ≥ 4.7 (含 ArUco 扩展模块)
数值计算	NumPy
串口通信	pyserial
云端大模型	API 接入 (用于题目解答任务)

表1

上位机与 MCU 之间采用基于文本行的轻量通信协议，以 `\n` 作为帧分隔符，支持以下指令：

- 1) `PING`：连接测试，MCU 回复 `PONG`。
 - 2) `RESET`：机械臂复位至初始姿态（末端坐标约 (15, 0, 2) cm）。
 - 3) `MOVE x y z pitch min\_p max\_p duration`：笛卡尔坐标运动控制，单位 cm/度/ms。
 - 4) `CALIB\_RESET`：触发手眼标定流程，机械臂移动至标定位置 01，固件回传实际关节角。
- 舵机总线底层采用帧协议（起始字节 0x55 0x55，含帧长、命令字、舵机数量、运动时间、ID 及目标位置），波特率 9600bps。

3.2 测试数据

3.2.1 手眼标定

手眼标定分三步完成：相机内参标定、标定数据采集与手眼变换矩阵求解。

相机内参标定：使用 9×6 棋盘格（格边长 2.4 cm），采集不少于 45 帧有效图像，调用 OpenCV 张正友标定法（`cv2.calibrateCamera`）计算内参矩阵 K 与畸变系数 $dist$ ，亚像素精化窗口 11×11 。

手眼数据采集：机械臂依次运动至 21 个预定义标定位姿，每个位姿采集一次 AprilTag 检测结果（`cv2.solvePnP` 估算标记→相机变换 $T_{\text{target2cam}}$ ）和正运动学末端→基座变换矩阵 T_{end2base} ，优先使用固件实时回传的实际关节角（避免机械传动间隙引入的误差）。

手眼矩阵求解：同时尝试 TSAI、HORAUD、ANDREFF、DANIILIDIS 四种算法，以旋转矩阵正交误差（ $\|R^T R - I\|$ ）为评价指标，选取最优解。测试数据如下表2所示：

评价指标	目标值	实测值
相机重投影误差	< 1.0 px	约 0.62 px
手眼一致性 RMS（标记在基座坐标系位置标准差）	< 0.5 cm（优）/ < 1.0 cm（可接受）	约 0.41 cm
最优算法	-	DANIILIDIS（对偶四元数法）

表2

3.2.2 逆运动学

逆运动学采用 ELBOW_UP 模式，连杆参数：基座高度 $L_1=2.89$ cm，大臂 $L_2=10.43$ cm，小臂 $L_3=8.90$ cm，腕部/末端杆 $L_4=17.70$ cm。对机械臂末端进行 10 组目标坐标的定点到达测试，记录实际末端偏差：

测试组	目标坐标 (x, y, z) cm	末端到达偏差(cm)	备注
1	(15, 0, 5)	± 0.3	正前方中心
2	(12, 5, 3)	± 0.4	右侧偏移
3	(10, -5, 4)	± 0.5	左侧偏移
4	(18, 0, 2)	± 0.3	远端低位
5	(8, 0, 8)	± 0.6	近端高位
6-10	多方向复合运动	$\pm 0.3 \sim 0.7$	—

表3

3.2.3 功能模块测试数据

颜色识别与分拣：在标准实验室光照条件下，分别测试红、绿、蓝三色物块识别，连续识别 30 次，统计识别准确率与分拣成功率。

颜色	识别准确率	分拣成功率	平均执行时间
红色	96.7%	93.3%	4.2s
绿色	93.3%	90.0%	4.5s
蓝色	96.7%	93.3%	4.1s

表4

唤醒词识别：在实验室环境（背景噪声约 45 dB）下，连续测试 50 次唤醒，唤醒成功率 92%，误唤醒率约 4%。

题目解答：对 20 道数学计算题（涵盖四则运算、方程等）通过摄像头拍摄后调用云端大模型解答，正确率 90%，平均响应时间（含网络传输）约 3.2 s。

人脸识别追踪：在正常室内光照下对 5 名测试者进行人脸检测，检测率 94%，当人员离座时报警响应时间约 0.8 s。

智能光照控制：语音指令识别后，灯光亮度调节响应时间约 1.1 s，亮度调节准确率（对应正确挡位）为 95%。

3.3 结果分析

实测逆运动学末端到达误差在 $\pm 0.3 \sim \pm 0.7$ cm 范围内，误差主要来源于以下三方面：

- 1) 舵机机械间隙：串口总线舵机的齿轮传动存在约 1° 以内的回程间隙，在远端（臂展较大）位置误差被放大。
- 2) 连杆弹性变形：大臂与小臂在承载夹爪和目标物重量时存在轻微弹性弯曲，在低位（俯角较大）时尤为明显。
- 3) 手眼标定残余误差：手眼一致性 RMS 约 0.41 cm，已优于 0.5 cm 的设计目标，但仍构成系统定位误差的底层上限。

整体而言，末端定位精度满足物块抓取（目标物直径约 3 cm）的工程要求。

颜色识别在光照条件良好时准确率稳定在 93%以上，但在强侧光或环境光偏色时识别率有所下降，建议实际部署时配合均匀补光灯以提升稳定性。手眼标定流程采用三重降级检测策略（去畸变+CLAHE 增强 $\rightarrow$ 原始+CLAHE $\rightarrow$ 原始灰度），有效应对复杂光照下 AprilTag 检测失败的情况，显著提升了标定数据采集的鲁棒性。

作品实现了“唤醒词 $\rightarrow$ 任务选择 $\rightarrow$ 执行 $\rightarrow$ 复位”的完整工作闭环，各功能模块间通过统一的任务调度框架协调，系统无明显竞争冲突。云端大模型接入在网络状况良好时响应时间满足交互需求（平均约 3.2 s），在弱网环境下存在超时风险，后续可考虑本地轻量化模型作为降级方案。综合测试结果表明，作品各项核心功能均达到设计目标，具备在实际教学辅助场景中稳定运行的能力。

4. 创新性说明

4.1 硬件创新

本作品在硬件层面进行了多项针对性设计，将机械臂与学习辅助场景深度融合：

1) 多传感器异构融合：在单一机械臂系统中集成了 USB 摄像头、超声波模块、OLED 显示屏等多类传感器，通过统一的任务调度框架实现协同工作，避免了传统方案中各传感器“各自为政”的问题。

2) 步进电机直线滑台扩展：在常规旋转关节基础上，引入 I²C 接口步进电机直线滑台（8 细分模式，最大 10400 步），为机械臂增加了线性位移维度，拓展了可达工作空间，支持更多元的操作场景。

3) 串口总线舵机统一管控：采用半双工串口总线方案替代传统 PWM 多路控制，所有舵机共用同一串口总线（ID 寻址），大幅简化了布线，并支持实时读取各舵机实际位置，为示教模式和手眼标定提供了真实反馈数据。

4.2 软件创新

1) 端云协同的任务执行架构：本地端负责实时运动控制和视觉处理，云端大模型负责自然语言理解与题目解答，形成“本地感知+云端智能”的分层架构，兼顾了响应实时性与智能化深度。

2) 唤醒词驱动的多任务调度：参照主流智能音箱的交互范式，设计了“唤醒词激活→大模型对话确认任务→执行→自动复位”的完整交互闭环，用户无需操作任何实体按钮，降低了使用门槛，提升了教学场景中的自然交互体验。

3) Web 实时状态可视化：通过 Web 界面实时展示机械臂当前姿态与即将执行的任务状态，便于教师和学生直观了解系统工作过程，兼具教学演示与故障诊断价值。

4.3 算法创新

1) 高精度手眼标定流水线：设计了“相机内参标定→多位姿数据采集→四算法竞优求解”三步流水线。在数据采集阶段，优先使用固件实时回传的舵机实际位置反推真实关节角，替代理论指令角，有效消除舵机传动间隙带来的系统误差；在求解阶段，同时运行 TSAI、HORAUD、ANDREFF、DANIILIDIS 四种主流手眼标定算法，以旋转矩阵正交误差为评价函数自动选取最优解，手眼一致性 RMS 达 0.41 cm，优于 0.5 cm 的设计目标。

2) 鲁棒 ArUco 检测策略：针对实际场景中光照不均的问题，设计了三重降级检测机制（去畸变+CLAHE 对比度增强 → 原始+CLAHE → 原始灰度），并综合使用自适应阈值窗口多尺度扫描（步长 10，窗口范围 3~23）与亚像素角点精化，显著提升了标记检测在复杂光照下的成功率。

3) 俯仰角自适应逆运动学：在标准 ELBOW_UP 逆运动学基础上，引入俯仰角范围双端搜索策略（PitchRange_Set），当用户指定俯仰角范围 [min_pitch, max_pitch] 时，算法依次尝试两端点与中间值，取第一个有效解，解决了特定坐标在固定俯仰角下无解的问题，提升了运动规划的鲁棒性。

5. 总结

本作品以 AI 机械臂为载体，面向学生学习辅助场景，设计并实现了集运动控制、视觉感知、语音交互与云端智能于一体的综合系统。

在硬件层面，以 Renesas RA6M5 (Cortex-M33) 为主控，通过串口总线统一管理 6 路舵机，配合 I²C 步进电机直线滑台拓展工作空间，集成 USB 摄像头、超声波、OLED 等多类传感器，构建了完整的感知-执行硬件平台。

在算法层面，实现了基于 DH 参数的 4 自由度正/逆运动学 (ELBOW_UP 模式)，配合俯仰角范围自适应搜索，保证了运动规划在全工作空间内的可行性；完成了张正友相机内参标定与四算法竞优手眼标定，末端视觉定位精度 (手眼一致性 RMS) 达 0.41 cm。

在功能层面，完整实现了颜色识别分拣、语音唤醒、题目解答、智能光照、人脸识别追踪、OLED 信息显示六大功能，各功能通过统一的唤醒词交互框架调度，形成完整的用户交互闭环。

附录

相机内参标定

"""

第一步：相机内参标定

你需要一张棋盘格图案（从网上打印，粘在硬纸板上）。

推荐尺寸：A4 纸，9x6 内角点棋盘格，每格约 25mm

操作步骤：

1. 把摄像头用 USB 接到电脑
2. 运行本脚本
3. 手持棋盘格，在摄像头前慢慢移动、倾斜、旋转
4. 当看到棋盘格角点被标出时，按 空格键 保存一帧
5. 多角度拍摄至少 45 张（越多越准）
6. 按 q 结束，自动计算并保存内参

输出文件：../model/calibration/camera_intrinsics.npz

"""

```
import cv2
```

```
import numpy as np
```

```
import os
```

```
from PIL import Image, ImageDraw, ImageFont
```

```
def _find_chinese_font() -> str:
```

```
    """查找系统中可用的中文字体路径"""
```

```
    candidates = [
```

```
        "C:/Windows/Fonts/msyh.ttc",          # 微软雅黑
```

```
        "C:/Windows/Fonts/simsun.ttc",        # 宋体
```

```
        "C:/Windows/Fonts/simhei.ttf",        # 黑体
```

```
        "/usr/share/fonts/truetype/wqy/wqy-zenhei.ttc",
```

```
        "/System/Library/Fonts/PingFang.ttc",
```

```
    ]
```

```
    for p in candidates:
```

```
        if os.path.exists(p):
```

```
            return p
```

```
    return ""
```

```
_FONT_PATH = _find_chinese_font()
```

```

def put_chinese_text(img, text: str, pos: tuple, font_size: int = 28,
                    color: tuple = (255, 255, 255)) -> np.ndarray:
    """在 OpenCV 图像上渲染中文文本（BGR 输入/输出）"""
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(img_rgb)
    draw = ImageDraw.Draw(pil_img)
    if _FONT_PATH:
        font = ImageFont.truetype(_FONT_PATH, font_size)
    else:
        font = ImageFont.load_default()
    # PIL 颜色为 RGB
    draw.text(pos, text, font=font, fill=(color[2], color[1], color[0]))
    return cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)

# —— 参数配置（根据你打印的棋盘格修改）——
BOARD_COLS    = 9          # 棋盘格横向内角点数
BOARD_ROWS    = 6          # 棋盘格纵向内角点数
SQUARE_SIZE   = 2.4        # 每格边长（cm）
CAMERA_INDEX  = 1          # 摄像头编号（一般为 0）
MIN_IMAGES    = 45         # 最少采集帧数
OUTPUT_DIR    = os.path.join(os.path.dirname(__file__), "../../model/calibration")
# ——

os.makedirs(OUTPUT_DIR, exist_ok=True)

criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
board_size = (BOARD_COLS, BOARD_ROWS)

# 棋盘格世界坐标（z=0 的平面）
objp = np.zeros((BOARD_COLS * BOARD_ROWS, 3), np.float32)
objp[:, :2] = np.mgrid[0:BOARD_COLS, 0:BOARD_ROWS].T.reshape(-1, 2) * SQUARE_SIZE

obj_points = [] # 世界坐标
img_points = [] # 图像坐标

cap = cv2.VideoCapture(CAMERA_INDEX)
if not cap.isOpened():
    print(f"[错误] 无法打开摄像头 {CAMERA_INDEX}，请检查连接")
    exit(1)

```

```

cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

# 摄像头反向安装（旋转 180°）
CAMERA_ROTATED_180 = True

print("=" * 55)
print("相机内参标定 —— 操作说明")
print("=" * 55)
print("    • 手持棋盘格在摄像头前缓慢移动")
print("    • 看到绿色角点时按 [空格] 保存")
print(f"    • 至少采集 {MIN_IMAGES} 张后按 [q] 计算内参")
print("=" * 55)

n_captured = 0

while True:
    ret, frame = cap.read()
    if not ret:
        print("[错误] 读取摄像头失败")
        break

    if CAMERA_ROTATED_180:
        frame = cv2.rotate(frame, cv2.ROTATE_180)

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    found, corners = cv2.findChessboardCorners(gray, board_size, None)

    display = frame.copy()
    if found:
        corners_sub = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria)
        cv2.drawChessboardCorners(display, board_size, corners_sub, found)
        display = put_chinese_text(display, "检测到棋盘格！按空格保存", (10, 8),
                                    font_size=28, color=(0, 255, 0))
    else:
        display = put_chinese_text(display, "未检测到棋盘格", (10, 8),
                                    font_size=28, color=(0, 0, 255))

    display = put_chinese_text(display, f"已采集: {n_captured} / {MIN_IMAGES} 按 q 结束",
                                (10, 44), font_size=24, color=(255, 255, 0))
    cv2.imshow("相机内参标定 - step1", display)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):

```

```

        break
    elif key == ord(' ') and found:
        corners_sub = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria)
        obj_points.append(objp)
        img_points.append(corners_sub)
        n_captured += 1
        print(f"  保存第 {n_captured} 张")
        # 闪烁提示
        cx = display.shape[1] // 2 - 56
        cy = display.shape[0] // 2 - 24
        display = put_chinese_text(display, "已保存!", (cx, cy), font_size=64,
                                    color=(0, 255, 255))
        cv2.imshow("相机内参标定 - step1", display)
        cv2.waitKey(500)

cap.release()
cv2.destroyAllWindows()

if n_captured < 5:
    print(f"[警告] 只采集了 {n_captured} 张，不足以标定，请重新运行")
    exit(1)

print(f"\n 开始计算内参（共 {n_captured} 张）...")
h, w = gray.shape
ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    obj_points, img_points, (w, h), None, None
)

# 计算重投影误差
total_err = 0.0
for i in range(len(obj_points)):
    projected, _ = cv2.projectPoints(obj_points[i], rvecs[i], tvecs[i], K, dist)
    err = cv2.norm(img_points[i], projected, cv2.NORM_L2) / len(projected)
    total_err += err
mean_err = total_err / len(obj_points)

print("\n===== 标定结果 =====")
print(f"相机内参矩阵 K:\n{K}")
print(f"畸变系数 dist:\n{dist}")
print(f"重投影误差（越小越好）: {mean_err:.4f} px")
if mean_err > 1.0:
    print("[提示] 误差较大，建议重新采集更多、更好的图片")
else:
    print("[OK] 误差在正常范围内")

```

```
# 保存
out_path = os.path.join(OUTPUT_DIR, "camera_intrinsics.npz")
np.savez(out_path, K=K, dist=dist, img_size=np.array([w, h]))
print(f"\n 内参已保存到: {out_path}")
```

机械臂控制

```
#include "hal_data.h"
#include "arm_control.h"
#include "serial_servo.h"
#include "comm.h"
#include "stdlib.h"
#include "string.h"
#include "stdio.h"

uint8_t g_status = 0;

/* —— 手眼标定姿态表（与 tools/algorithm/step2_collect_data.py 中 CALIB_POSES 完全一致）
—— */

typedef struct { float x; float y; float z; float pitch; } CalibPose_t;

#define CALIB_POSES_COUNT 21
static const CalibPose_t s_calib_poses[CALIB_POSES_COUNT] = {
    { 16.0f, 0.0f, -3.2f, -76.1f }, /* 01 中心 dist=16.32 */
    { 17.0f, -3.0f, -3.2f, -76.1f }, /* 02 x+1 y-3 dist=17.56 */
    { 18.0f, 0.0f, -3.2f, -76.1f }, /* 03 x+2 dist=18.28 */
    { 16.0f, -3.0f, -3.2f, -76.1f }, /* 04 y-3 dist=16.59 */
    { 16.0f, 3.0f, -3.2f, -76.1f }, /* 05 y+3 dist=16.59 */
    { 17.0f, 0.0f, -1.0f, -71.0f }, /* 06 x+1 z+2.2 pitch+5 dist=17.03 */
    { 16.0f, 0.0f, -5.5f, -76.1f }, /* 07 z-2.3 dist=16.92 */
    { 16.0f, 0.0f, -3.2f, -71.0f }, /* 08 pitch+5 dist=16.32 */
    { 16.0f, 0.0f, -3.2f, -81.0f }, /* 09 pitch-5 dist=16.32 */
    { 17.0f, 3.0f, -3.2f, -76.1f }, /* 10 x+1 y+3 dist=17.56 */
    { 17.0f, 0.0f, -5.5f, -76.1f }, /* 11 x+1 z-2.3 dist=17.87 */
    { 18.0f, -3.0f, -3.2f, -76.1f }, /* 12 x+2 y-3 dist=18.56 */
    { 18.0f, 3.0f, -3.2f, -76.1f }, /* 13 x+2 y+3 dist=18.56 */
    { 18.0f, -3.0f, -1.0f, -71.0f }, /* 14 x+2 y-3 z+2.2 pitch+5 dist=18.28 */
    { 18.0f, 0.0f, -5.5f, -81.0f }, /* 15 x+2 z-2.3 pitch-5 dist=18.82 */
    { 16.0f, -3.0f, -1.0f, -71.0f }, /* 16 y-3 z+2.2 pitch+5 dist=16.31 */
    { 16.0f, 3.0f, -5.5f, -81.0f }, /* 17 y+3 z-2.3 pitch-5 dist=16.92 */
    { 17.0f, -3.0f, -5.5f, -81.0f }, /* 18 x+1 y-3 z-2.3 pitch-5 dist=18.12 */
    { 18.0f, 3.0f, -1.0f, -71.0f }, /* 19 x+2 y+3 z+2.2 pitch+5 dist=18.28 */
    { 17.0f, 3.0f, -1.0f, -71.0f }, /* 20 x+1 y+3 z+2.2 pitch+5 dist=17.29 */
    { 18.0f, -3.0f, -5.5f, -81.0f }, /* 21 x+2 y-3 z-2.3 pitch-5 dist=18.82 */
}
```

```

};

/* 当前标定位置下标 (CALIB_RESET 后从 1 开始, 因 RESET 已执行位置 0) */
static uint8_t s_calib_idx = 0;

/**
 * @brief 驱动机械臂到指定标定位置, 等待完成后读取实际关节角并回传。
 *
 * 回传格式 (含 '\n') :
 * 成功且读位置成功: "OK j0.dd,j1.dd,j2.dd,j3.dd\n" (实际关节角, 单位度)
 * 成功但读位置失败: "OK j0.dd,j1.dd,j2.dd,j3.dd\n" (回退到运动学命令角)
 * 运动学无解: "ERR\n"
 */
static void s_calib_move_and_report(const CalibPose_t *pose)
{
    char reply[72];
    uint16_t positions[ARM_MAX_SERVOS_NUM] = {0};
    float joints[4] = {0};

    /* 1. 计算 IK 并驱动舵机 */
    g_status = ArmControl_CoordinateSet(
        pose->x, pose->y, pose->z, pose->pitch, -90.0f, 90.0f, 1000);

    if (!g_status)
    {
        Comm_SendStr("ERR\n");
        return;
    }

    /* 2. 等待运动完成 (duration=1000ms, 再留 500ms 余量) */
    R_BSP_SoftwareDelay(1500, BSP_DELAY_UNITS_MILLISECONDS);

    /* 3. 读取舵机实际位置 → 关节角
     * 成功: 使用真实反馈; 失败: 回退到运动学命令角 (g_kin_obj) */
    if (ArmControl_ReadAllPositions(positions, 500))
    {
        ArmControl_PositionsToJointAngles(positions, joints);
    }
    else
    {
        /* 读取失败, 使用 CoordinateSet 内部存储的运动学解作为后备 */
        joints[0] = g_kin_obj.joint[0].theta;
        joints[1] = g_kin_obj.joint[1].theta;
        joints[2] = g_kin_obj.joint[2].theta;
    }
}

```

```

        joints[3] = g_kin_obj.joint[3].theta;
    }

    /* 4. 格式化回传: OK j0.dd,j1.dd,j2.dd,j3.dd\n */
    snprintf(reply, sizeof(reply),
             "OK %.2f,%.2f,%.2f,%.2f\n",
             joints[0], joints[1], joints[2], joints[3]);
    Comm_SendStr(reply);
}

void hal_entry(void)
{
    // 1. 打开串口 (舵机通信)
    R_SCI_UART_Open(&g_serial_servo_uart_ctrl, &g_serial_servo_uart_cfg);

    // 2. 初始化机械臂控制
    ArmControl_Init();

#ifdef PC_CONTROL
    // 3. 初始化 PC 通信串口 (g_com_uart)
    Comm_Init();
#endif

    // 等待系统稳定
    R_BSP_SoftwareDelay(3000, BSP_DELAY_UNITS_MILLISECONDS);

    /* ===== 示教模式: 卸力后用手掰动机械臂, 实时观察末端坐标 =====
     * 在 Keil 调试器的 Watch 窗口添加 g_teach_data 即可查看实时 XYZ。
     * 每 100ms 读取一次舵机位置并计算正向运动学。
     */
    // ArmControl_TeachMode(100);

#ifdef PC_CONTROL
    /*
     * PC 控制模式: 循环读取上位机发来的文本行并执行
     *
     * 支持的指令 (均以 \n 或 \r\n 结尾):
     *
     *   PING
     *       → 返回 "PONG\n", 用于连接测试
     *
     *   MOVE x y z pitch min_pitch max_pitch duration
     *       → 调用 ArmControl_CoordinateSet, 返回 "OK\n" 或 "ERR\n"
     *       → 示例: MOVE 16 0 -3.2 -76.1 -90 90 1000
     */

```

```

*
*   RESET
*       → 机械臂复位到出厂初始位置，返回 "OK\n"
*
*   CALIB_RESET
*       → 手眼标定：重置序号，机械臂移动到位置 01（中心），返回 "OK\n" 或 "ERR\n"
*       → 上位机在启动 step2_collect_data.py 时自动发送
*
*   1
*       → 手眼标定：机械臂移动到下一个标定位置，返回 "OK\n" 或 "ERR\n"
*       → 全部 21 个位置执行完后返回 "DONE\n"
*       → 上位机每次按空格采集完数据后自动发送
*/
char line[COMM_RX_BUF_SIZE];

while (1)
{
    if (!Comm_HasLine())
        continue;

    Comm_ReadLine(line, sizeof(line));

    if (strcmp(line, "PING") == 0)
    {
        Comm_SendStr("PONG\n");
    }
    else if (strcmp(line, "RESET") == 0)
    {
        ArmControl_Reset(&g_arm_ctrl, 1000);
        Comm_SendStr("OK\n");
    }
    else if (strncmp(line, "MOVE ", 5) == 0)
    {
        char *p = line + 5;
        char *end = p;

        float x = strtod(p, &end); p = end + 1;
        float y = strtod(p, &end); p = end + 1;
        float z = strtod(p, &end); p = end + 1;
        float pitch = strtod(p, &end); p = end + 1;
        float min_p = strtod(p, &end); p = end + 1;
        float max_p = strtod(p, &end); p = end + 1;
        uint16_t duration = (uint16_t)strtoul(p, NULL, 10);
    }
}

```

```

        g_status = ArmControl_CoordinateSet(x, y, z, pitch, min_p, max_p, duration);
        Comm_SendStr(g_status == 0 ? "OK\n" : "ERR\n");
    }
    /*
    * 手眼标定专用指令
    *
    * CALIB_RESET
    *     → 重置位置序号，机械臂移动到第 1 个标定位置（中心），返回 "OK\n"
    *
    * 1
    *     → 机械臂移动到下一个标定位置，返回 "OK\n"
    *     → 所有位置已完成时返回 "DONE\n"
    */
    else if (strcmp(line, "CALIB_RESET") == 0)
    {
        /* 重置序号，移动到位置 01（中心），回传实际关节角 */
        s_calib_idx = 0;
        s_calib_move_and_report(&s_calib_poses[s_calib_idx++]);
    }
    else if (strcmp(line, "1") == 0)
    {
        if (s_calib_idx >= CALIB_POSES_COUNT)
        {
            Comm_SendStr("DONE\n");
        }
        else
        {
            /* 移动到下一个标定位置，回传实际关节角 */
            s_calib_move_and_report(&s_calib_poses[s_calib_idx++]);
        }
    }
}

#else
/* ===== 常规运动模式（与示教模式互斥，需注释掉上面的 TeachMode） ===== */
while(1)
{
    // g_status = ArmControl_CoordinateSet(16,0,(float)-3.2,(float)-76.1,-90,90,1000);
    // R_BSP_SoftwareDelay(2000, BSP_DELAY_UNITS_MILLISECONDS);
    // g_status = ArmControl_CoordinateSet(15,-6,20,0,-90,90,1000);
    // R_BSP_SoftwareDelay(2000, BSP_DELAY_UNITS_MILLISECONDS);
    // g_status = ArmControl_CoordinateSet(30,0,13,15,-90,90,1000);
    // R_BSP_SoftwareDelay(2000, BSP_DELAY_UNITS_MILLISECONDS);
    // g_status = ArmControl_CoordinateSet(30,0,13,30,-90,90,1000);

```

```

//      R_BSP_SoftwareDelay(2000, BSP_DELAY_UNITS_MILLISECONDS);
    }
#endif

#if BSP_TZ_SECURE_BUILD
    /* Enter non-secure code */
    R_BSP_NonSecureEnter();
#endif
}

#if BSP_TZ_SECURE_BUILD

FSP_CPP_HEADER
BSP_CMSE_NONSECURE_ENTRY void template_nonsecure_callable ();

/* Trustzone Secure Projects require at least one nonsecure callable function in order to build (Remove
this if it is not required to build). */
BSP_CMSE_NONSECURE_ENTRY void template_nonsecure_callable ()
{

}

FSP_CPP_FOOTER

#endif

```

参考文献：

[1] TSAI R Y, LENZ R K. A new technique for fully autonomous and efficient 3D robotics hand/eye calibration (一种全自主高效机器人三维手眼标定新方法) [J]. IEEE Transactions on Robotics and Automation, 1989, 5(3): 345-358.

[2] ZHANG Z. A flexible new technique for camera calibration (一种灵活的摄像机标定新技术) [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000, 22(11): 1330-1334.

[3] GARRIDO-JURADO S, MUÑOZ-SALINAS R, MADRID-CUEVAS F J, et al. Automatic generation and detection of highly reliable fiducial markers under occlusion (遮挡条件下高可靠基准标记的自动生成与检测) [J]. Pattern Recognition, 2014, 47(6): 2280-2292.

- [4] 刘极峰, 易际明. 机器人技术基础[M]. 2版. 北京: 高等教育出版社, 2019: 56-134.
- [5] CRAIG J J. Introduction to Robotics: Mechanics and Control (机器人学导论: 力学与控制) [M]. 3rd ed. Upper Saddle River: Pearson Prentice Hall, 2005: 63-201.
- [6] 彭秀艳, 李晔, 赵玉新. 基于改进 HSV 颜色空间的目标颜色识别方法 (Improved target color recognition method based on HSV color space) [J]. 哈尔滨工程大学学报, 2010, 31(3): 368-373.
- [7] DE ANDRÉS-TORO B, GIRON-SIERRA J M, LOPEZ-OROZCO J A, et al. A kinematic robot arm control optimization using extended Kalman filter (基于扩展卡尔曼滤波的机械臂运动控制优化) [J]. Robotics and Autonomous Systems, 1998, 23(4): 221-232.
- [8] 孙富春, 孙增圻, 张再兴. 智能机器人学[M]. 北京: 清华大学出版社, 2004: 102-178.
- [9] OpenCV开发团队. OpenCV官方文档: ArUco marker detection[EB/OL]. [2025-03-10]. https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html.